

JavaScript Debugging Console

[Back to JS page](#)

Table of Contents

- [JavaScript Debugging Console](#)
 - [Using console.log\(\).](#)
 - [Example 1](#)
 - [Using console.warn\(\).](#)
 - [Example 2](#)
 - [Using console.error\(\).](#)
 - [Example 3](#)
 - [Logging Multiple Values](#)
 - [Example 4](#)
 - [Inspecting Objects](#)
 - [Example 5](#)
 - [Using console.table\(\).](#)
 - [Example 6](#)
 - [Debugging Tip: Stop Guessing](#)
 - [Example 7](#)
 - [Document](#)
 - [Reference](#)

Using console.log()

The `console.log()` method is the most common console method. Use it to print values and see what your program is doing. It prints messages and can print variable values to the browser's console.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Console</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using console.log()</h4>
<p>Open the browser console (F12) to see the debug output.</p>
<p id="demo"></p>

<script>
let name = "John";
let age = 25;

console.log("Hello from JavaScript!");
console.log(name);
console.log(age);

document.getElementById("demo").innerHTML =
  "name = " + name + ", age = " + age + " (check console)";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Using console.warn()

Use `console.warn()` to display warnings. Warnings tell you something might be wrong, but your code can still run. Use warnings when you want to highlight something suspicious in your program.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Console</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using console.warn()</h4>
<p>Open the browser console (F12) to see warning messages.</p>
<p id="demo"></p>

<script>
let age = 15;

if (age < 18) {
  console.warn("This is a warning! User is under 18 years old.");
  console.warn("Age: " + age);
}

document.getElementById("demo").innerHTML =
  "Warning example: age = " + age + " (check console for warning)";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Using console.error()

Use `console.error()` to display error messages. Errors show that something has failed. The error message is displayed in red in most browsers' consoles, making it easy to spot.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Console</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using console.error()</h4>
<p>Open the browser console (F12) to see error messages.</p>
<p id="demo"></p>

<script>
let text = "";

try {
  // Intentionally cause an error
  undefinedFunction();
}
catch(err) {
  console.error("Something went wrong!");
  console.error("Error: " + err.message);
  text = "Error caught: " + err.message;
}

document.getElementById("demo").innerHTML = text + " (check console)";
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Logging Multiple Values

You can log more than one value at once with `console.log()`. Simply separate values with commas. This is very useful for tracking related variable values together.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Console</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Logging Multiple Values</h4>
<p>Open the browser console (F12) to see the debug output.</p>
<p id="demo"></p>

<script>
let x = 10;
let y = 5;
let sum = x + y;

console.log("x =", x, "y =", y);
console.log("x =", x, "y =", y, "sum =", sum);

document.getElementById("demo").innerHTML =
  "x = " + x + ", y = " + y + ", sum = " + sum + " (check console)";
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Inspecting Objects

The console is very useful for inspecting objects. You can log an object with `console.log()`, then in the console, you can click the object to expand it and see its properties.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Console</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Inspecting Objects</h4>
<p>Open the browser console (F12) and expand the object to see its properties.</p>
<p id="demo"></p>

<script>
let user = {
  name: "John",
  age: 25,
  email: "john@example.com",
  city: "New York"
};

console.log(user);
console.log("User name:", user.name);
console.log("User age:", user.age);

document.getElementById("demo").innerHTML =
  "User: " + user.name + ", Age: " + user.age + " (check console to inspect object)";
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Using console.table()

Use `console.table()` to display data in a table format. Tables make it much easier to debug arrays of objects. The data is displayed as a formatted table in the browser console.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Console</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using console.table()</h4>
<p>Open the browser console (F12) to see the table output.</p>
<p id="demo"></p>

<script>
let users = [
  {name: "John", age: 25, city: "New York"},
  {name: "Anna", age: 30, city: "London"},
  {name: "Peter", age: 28, city: "Paris"}
];

console.table(users);
console.log("Users array logged as table (check console)");

let displayText = "";
for (let user of users) {
  displayText += user.name + " (" + user.age + ") - " + user.city + "<br>";
}
document.getElementById("demo").innerHTML =
  displayText + "(check console for table view)";
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Debugging Tip: Stop Guessing

Beginners often **guess** what the value is. Professionals **log the value** and confirm it. If your code is not working, **do not guess**. Use `console.log()`. Every time you assume a value, log it to the console and verify.

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Debugging Console</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Debugging Tip: Stop Guessing</h4>
<p>Open the browser console (F12) to see the debug output.</p>
<p id="demo"></p>

<script>
let result = "";

// Don't guess what the value is – log it!
let a = 5;
let b = "10";

console.log("Value of a:", a, "| type:", typeof a);
console.log("Value of b:", b, "| type:", typeof b);

let total = a + b;
console.log("total = a + b =", total, "| type:", typeof total);

// Now fix it by converting
let fixedTotal = a + Number(b);
console.log("fixedTotal = a + Number(b) =", fixedTotal, "| type:", typeof fixedTotal);

result += "a = " + a + " (type: " + typeof a + ")<br>";
result += "b = \"\" + b + "\" (type: " + typeof b + ")<br>";
result += "a + b = " + total + " (unexpected string concatenation!)<br>";
result += "a + Number(b) = " + fixedTotal + " (correct addition)";

document.getElementById("demo").innerHTML = result;
</script>

</body>
</html>

```

Example 7

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Debugging Console](#)